

Software lab

🕒 Created	May 27, 2026 11:02 AM
☰ Tags	

CSE 474 Exam Task Manager

1 Git Version Control Tasks

Basic Commands Practice

- ✓ Install & verify Git (`git --version`)
- ✓ Initialize repository → `git init`
- ✓ Add files → `git add`
- ✓ Commit changes → `git commit -m ""`
- ✓ Check status → `git status`
- ✓ View history → `git log`

Branching & Merge

- ✓ Create branch → `git branch`
- ✓ Switch branch → `git checkout`
- ✓ Create + switch → `git checkout -b`
- ✓ Merge branches → `git merge`

Remote Repository

- ✓ Connect GitHub repo
- ✓ Push code → `git push`
- ✓ Pull code → `git pull`

☒ Clone repo → `git clone`

2 API Development & Postman Tasks

REST Methods Practice

- ☒ GET request
- ☒ POST request
- ☒ PUT request
- ☒ PATCH request
- ☒ DELETE request

Postman

- ☐ Create collection
- ☐ Send requests
- ☐ Test endpoints
- ☐ Add headers/body
- ☐ JSON response handling
- ☐ Environment variables practice

Coding Practice

- ☐ Python requests module
 - ☐ API integration code
 - ☐ Error handling
 - ☐ Status code checking
-

3 Selenium IDE Tasks

- ☐ Install Selenium IDE
- ☐ Record test case

- ☐ Edit steps
- ☐ Add assertions
- ☐ Run automation
- ☐ Export scripts

Practice:

- Login automation
- Form submission
- Search automation

4 Selenium WebDriver Tasks

Environment Setup:

- ☐ Selenium installed
- ☐ ChromeDriver setup
- ☐ Firefox driver setup

Coding:

- ☐ Open browser
- ☐ Locate elements
 - id
 - xpath
 - css selector
- ☐ Click actions
- ☐ Send keys
- ☐ Explicit wait
- ☐ Handle alerts
- ☐ Close browser

Mini Projects:

- ☐ Login automation
 - ☐ Search automation
 - ☐ Form testing
-

5 Pytest Framework Tasks

- ☐ Install pytest
- ☐ Create test files
- ☐ Assertions practice
- ☐ Execute tests → `pytest`
- ☐ Run specific tests
- ☐ Fixtures
- ☐ Test reports

Practice:

```
def test_sum():  
    assert 2+2 == 4
```

6 Environment Setup Checklist

Before exam:

- ☐ VS Code / IDE ready
- ☐ Python configured
- ☐ Postman installed
- ☐ Git working
- ☐ Chrome installed
- ☐ Node installed
- ☐ ChromeDriver matched

- ☐ Selenium installed
- ☐ Pytest installed
- ☐ Internet checked
- ☐ GitHub login ready

Commands:

```
pip install selenium
pip install pytest
pip install requests
git --version
pytest --version
node -v
```

GitHub-এ একটি project upload করার জন্য ধাপগুলো:

Step 1: Project folder-এ Git initialize করুন

```
git init
```

Step 2: সব file staging area-তে যোগ করুন

```
git add .
```

Step 3: Git user information configure করুন (শুধুমাত্র প্রথমবার)

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Example:

```
git config --global user.name "Saniul Islam"
git config --global user.email "your_github_email@example.com"
```

Step 4: Initial commit তৈরি করুন

```
git commit -m "Initial commit"
```

Step 5: Main branch set করুন

```
git branch -M main
```

Step 6: GitHub repository-এর সাথে connect করুন

```
git remote add origin https://github.com/isaniul333-abc/basic-working-checkings.git
```

Step 7: Code GitHub-এ push করুন

```
git push -u origin main
```

সব command একসাথে

```
git init
git add .
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/isaniul333-abc/basic-working-checkings.git
git push -u origin main
```

Note: `Your Name` এবং `your_email@example.com` এর জায়গায় আপনার GitHub account-এর নাম ও email ব্যবহার করবেন।

1 Git Version Control Tasks

Basic Commands Practice

Install & Verify Git

```
git --version
```

Initialize Repository

```
git init
```

Add Files

সব file add করতে:

```
git add .
```

নির্দিষ্ট file add করতে:

```
git add filename.txt
```

Commit Changes

```
git commit -m "Initial commit"
```

Check Status

```
git status
```

View History

```
git log
```

সংক্ষিপ্ত history:

```
git log --oneline
```

2 Branching & Merge

✓ Create Branch

```
git branch feature-branch
```

সব branch দেখুন:

```
git branch
```

✓ Switch Branch

```
git checkout feature-branch
```

✓ Create + Switch Branch

```
git checkout -b feature-branch
```

✓ Merge Branches

প্রথমে main branch-এ যান:

```
git checkout main
```

তারপর merge করুন:


```
git merge feature-branch
```

3 Remote Repository

✓ Connect GitHub Repository

```
git remote add origin https://github.com/USERNAME/REPOSITORY.  
git
```

Remote check:

```
git remote -v
```

✓ Push Code

প্রথমবার:

```
git push -u origin main
```

পরবর্তীতে:

```
git push
```

✓ Pull Code

```
git pull origin main
```

অথবা:

```
git pull
```

✓ Clone Repository

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

Complete Practice Flow

```
git --version
```

```
git init
```

```
git add .
```

```
git status
```

```
git commit -m "Initial commit"
```

```
git log --oneline
```

```
git checkout -b feature-branch
```

```
git branch
```

```
git checkout main
```

```
git merge feature-branch
```

```
git remote add origin https://github.com/isaniul333-abc/basic-working-checkings.git
```

```
git push -u origin main
```

```
git pull origin main
```

```
git clone https://github.com/isaniul333-abc/basic-working-checkings.git
```

Git Branch Workflow (Feature Branch → Main Branch)

Step 1: Feature Branch এ যাও

```
git checkout feature-branch
```

Purpose: Feature branch এ কাজ করার জন্য branch switch করা।

Step 2: Changes Stage করো

```
git add .
```

Purpose: Modified এবং New files staging area-তে যোগ করা।

Step 3: Commit করো

```
git commit -m "Added React CRUD API project"
```

Purpose: Changes local repository তে save করা।

Step 4: Feature Branch GitHub এ Push করো

```
git push -u origin feature-branch
```

Purpose: Feature branch remote repository তে upload করা।

Step 5: Main Branch এ যাও

```
git checkout main
```

Purpose: Main branch এ switch করা।

Step 6: Latest Main Pull করো

```
git pull origin main
```

Purpose: GitHub এর latest main branch sync করা।

Step 7: Feature Branch Merge করো

```
git merge feature-branch
```

Purpose: Feature branch এর changes main branch এ যুক্ত করা।

Step 8: Updated Main GitHub এ Push করো

```
git push origin main
```

Purpose: Merge করা code GitHub main branch এ upload করা।

Complete Workflow

```
git checkout feature-branch  
  
git add .  
  
git commit -m "Added React CRUD API project"  
  
git push -u origin feature-branch  
  
git checkout main  
  
git pull origin main  
  
git merge feature-branch
```

```
git push origin main
```

Optional Cleanup

Local Feature Branch Delete

```
git branch -d feature-branch
```

Remote Feature Branch Delete

```
git push origin --delete feature-branch
```

Workflow Diagram

Feature Branch

↓

git add .

↓

git commit

↓

git push origin feature-branch

↓

Main Branch

↓

git pull origin main

↓

git merge feature-branch

↓

git push origin main

↓

GitHub Main Updated

এই নোটটা সরাসরি Notion-এ paste করলে সুন্দর Git Workflow documentation হিসেবে থাকবে।

2 API Development

First Check Node Js installed or not ?

```
node -v
```

React (JavaScript)

```
npm create vite@latest my-react-app -- --template react
```

1. React project create করো

```
npm create vite@latest my-react-app -- --template react
```

2. Tailwind install করো

```
npm install tailwindcss @tailwindcss/vite  
npm install axios
```

3. vite.config.js edit করো

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'
```

```
import tailwindcss from '@tailwindcss/vite' // this line pest

export default defineConfig({
  plugins: [react(), tailwindcss()], //just tailwind() plugin
  s a ...
})
```

4. `src/index.css` replace করো

সব content delete করে লিখো:

```
@import "tailwindcss";
```

5. `src/App.jsx` test করো

```
import React, { useEffect, useState } from "react";
import axios from "axios";

const API = "https://jsonplaceholder.typicode.com/users";

export default function App() {
  // =====
  // 1. STATES SECTION (স্টেটসমূহ)
  // =====

  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [editingUserId, setEditingUserId] = useState(null);
  const [editMethod, setEditMethod] = useState(null);

  const [formData, setFormData] = useState({
    name: "",
    username: "",
```

```

        email: "",
        phone: "",
        website: "",
    });

    // =====
    // 2. HELPER FUNCTIONS (সহায়ক ফাংশনসমূহ)
    // =====
    // ফর্ম ক্লিয়ার করার ফাংশন
    const resetForm = () => {
        setFormData({ name: "", username: "", email: "", phone:
"", website: "" });
        setEditingUserId(null);
        setEditMethod(null);
    };

    // এডিট মোড শুরু করার ফাংশন
    const startEdit = (user, method) => {
        setEditingUserId(user.id);
        setEditMethod(method);

        if (method === "PATCH") {
            setFormData({
                name: user.name,
                username: "",
                email: "",
                phone: "",
                website: "",
            });
        } else {
            setFormData({
                name: user.name,
                username: user.username || "",

```



```

        email: user.email || "",
        phone: user.phone || "",
        website: user.website || "",
    });
}
window.scrollTo({ top: 0, behavior: "smooth" });
};

// =====
// 3. API METHODS SECTION (GET, POST, PUT, PATCH, DELETE)
// =====

// --- [PART: GET - Fetch Users] ---
const fetchUsers = async () => {
    setLoading(true);
    try {
        const res = await axios.get(API);
        setUsers(res.data);
    } catch (error) {
        console.log(error);
    }
    setLoading(false);
};

useEffect(() => {
    fetchUsers();
}, []);

// --- [PART: POST - Create User] ---
const handleCreate = async () => {
    if (!formData.name || !formData.username || !formData.email) {
        alert("Please fill required fields (Name, Username, Email)");
    }
};

```

```

il)");
    return;
  }
  try {
    const res = await axios.post(API, formData);
    setUsers([
      { ...res.data, id: Date.now() },
      ...users
    ]);
    resetForm();
    alert("🟢 User Created Successfully (POST Success)");
  } catch (error) {
    console.log(error);
  }
};

// --- [PART: PUT - Full Update] ---
const handlePutUpdate = async () => {
  if (!formData.name || !formData.username || !formData.email) {
    alert("PUT requires all mandatory fields to be filled!");
    return;
  }
  try {
    const targetId = editingUserId > 10 ? 1 : editingUserId;

    const res = await axios.put(`${API}/${targetId}`, {
      id: editingUserId,
      ...formData,
    });

    setUsers(
      users.map((user) => (user.id === editingUserId ? { ...res.data, id: editingUserId } : user))
    );
    resetForm();
    alert("🟡 Whole User Replaced Successfully (PUT Success)");
  } catch (error) {
    console.log(error);
  }
}

```

```

};

// --- [PART: PATCH - Partial Update] ---
const handlePatchUpdate = async () => {
  if (!formData.name) {
    alert("Please enter a name to patch!");
    return;
  }
  try {
    const targetId = editingUserId > 10 ? 1 : editingUserI
d;
    const res = await axios.patch(`${API}/${targetId}`, { n
ame: formData.name });
    setUsers(users.map((user) => user.id === editingUserId
? { ...user, name: res.data.name } : user));
    resetForm();
    alert("🟡 User Name Patched Successfully (PATCH Succes
s)");
  } catch (error) {
    console.log(error);
  }
};

// --- [PART: DELETE - Remove User] ---
const handleDelete = async (id) => {
  if (!window.confirm("Are you sure you want to delete this
user?")) return;
  try {
    const targetId = id > 10 ? 1 : id;
    await axios.delete(`${API}/${targetId}`);
    setUsers(users.filter((user) => user.id !== id));
    alert("🔴 User Deleted Successfully");
  } catch (error) {
    console.log(error);
  }
};

```

```

// =====
=====
// 4. RENDERING SECTION (UI / JSX)
// =====
=====
return (
  <div className="min-h-screen bg-gradient-to-tr from-slate-50 via-slate-100 to-blue-50 p-4 md:p-8 font-sans antialiased">

    {/* --- [UI PART: HEADER] --- */}
    <div className="max-w-6xl mx-auto text-center mb-12">
      <h1 className="text-4xl md:text-5xl font-black tracking-tight mb-3 bg-clip-text text-transparent bg-gradient-to-r from-blue-600 to-indigo-600">
        User Management Dashboard
      </h1>
      <p className="text-slate-500 font-medium max-w-md mx-auto">
        Manage system users, test HTTP REST methods (GET, POST, PUT, PATCH, DELETE) seamlessly.
      </p>
    </div>

    {/* --- [UI PART: DYNAMIC FORM] --- */}
    <div className={`max-w-4xl mx-auto bg-white/80 backdrop-blur-md rounded-3xl border shadow-xl p-6 md:p-8 mb-12 transition-all duration-500 ${
      editMethod === 'PUT' ? 'border-amber-400 ring-2 ring-amber-400/20' :
      editMethod === 'PATCH' ? 'border-indigo-400 ring-2 ring-indigo-400/20' : 'border-slate-200/60'
    }`>
      <div className="flex items-center justify-between mb-

```

```

6">
    <div className="flex items-center gap-3">
        <div className={`p-2.5 rounded-xl font-bold text-
white ${
            editMethod === 'PUT' ? 'bg-amber-500' : editMet
hod === 'PATCH' ? 'bg-indigo-500' : 'bg-blue-600'
        }`} >
            {editMethod === 'PUT' ? '✎' : editMethod ===
'PATCH' ? '🔧' : '➕'}
        </div>
        <h2 className="text-2xl font-bold text-slate-80
0">
            {editMethod === 'PUT' ? 'Edit User (PUT Mode -
Complete Update)' :
                editMethod === 'PATCH' ? 'Patch User (PATCH Mo
de - Partial Update)' : 'Add New System User'}
        </h2>
    </div>
    {editingUserId && (
        <button onClick={resetForm} className="text-sm bg
-slate-200 hover:bg-slate-300 text-slate-700 px-3 py-1.5 roun
ded-lg font-medium transition-all">
            Cancel Edit
        </button>
    )}
</div>

{/* INPUT FIELDS CONTAINER */}
<div className="grid md:grid-cols-2 gap-5">
    <div className="flex flex-col gap-1.5">
        <label className="text-xs font-bold text-slate-60
0 uppercase tracking-wider">Full Name *</label>
        <input
            type="text"
            placeholder="Enter Name"
            value={formData.name}

```

```

        onChange={(e) => setFormData({ ...formData, name: e.target.value })}
        className="border border-slate-200 p-3 rounded-xl bg-slate-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-blue-500/20 focus:border-blue-500 transition-all text-slate-700"
      />
    </div>

    <div className="flex flex-col gap-1.5">
      <label className="text-xs font-bold text-slate-600 uppercase tracking-wider">Username {editMethod !== 'PATCH' && '*'}</label>
      <input
        type="text"
        placeholder={editMethod === 'PATCH' ? 'Disabled in PATCH' : 'Enter username'}
        disabled={editMethod === 'PATCH'}
        value={formData.username}
        onChange={(e) => setFormData({ ...formData, username: e.target.value })}
        className="border border-slate-200 p-3 rounded-xl bg-slate-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-blue-500/20 focus:border-blue-500 transition-all text-slate-700 disabled:opacity-50 disabled:bg-slate-100"
      />
    </div>

    <div className="flex flex-col gap-1.5">
      <label className="text-xs font-bold text-slate-600 uppercase tracking-wider">Email Address {editMethod !== 'PATCH' && '*'}</label>
      <input
        type="email"
        placeholder={editMethod === 'PATCH' ? 'Disabled in PATCH' : 'something@gmail.com'}

```

```

        disabled={editMethod === 'PATCH'}
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
        className="border border-slate-200 p-3 rounded-xl bg-slate-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-blue-500/20 focus:border-blue-500 transition-all text-slate-700 disabled:opacity-50 disabled:bg-slate-100"
      />
    </div>

    <div className="flex flex-col gap-1.5">
      <label className="text-xs font-bold text-slate-600 uppercase tracking-wider">Phone Number</label>
      <input
        type="text"
        placeholder={editMethod === 'PATCH' ? 'Disabled in PATCH' : '+8801*****'}
        disabled={editMethod === 'PATCH'}
        value={formData.phone}
        onChange={(e) => setFormData({ ...formData, phone: e.target.value })}
        className="border border-slate-200 p-3 rounded-xl bg-slate-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-blue-500/20 focus:border-blue-500 transition-all text-slate-700 disabled:opacity-50 disabled:bg-slate-100"
      />
    </div>

    <div className="flex flex-col gap-1.5 md:col-span-2">
      <label className="text-xs font-bold text-slate-600 uppercase tracking-wider">Website URL</label>
      <input
        type="text"
        placeholder={editMethod === 'PATCH' ? 'Disabled

```

```

in PATCH' : 'www.websiteaddress.com'}
    disabled={editMethod === 'PATCH'}
    value={formData.website}
    onChange={(e) => setFormData({ ...formData, website: e.target.value })}
    className="border border-slate-200 p-3 rounded-xl bg-slate-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-blue-500/20 focus:border-blue-500 transition-all text-slate-700 disabled:opacity-50 disabled:bg-slate-100"
  />
</div>
</div>

{/* DYNAMIC ACTION BUTTONS */}
<div className="flex justify-end mt-6">
  {editMethod === "PUT" ? (
    <button
      onClick={handlePutUpdate}
      className="w-full md:w-auto bg-gradient-to-r from-amber-500 to-orange-600 text-white font-semibold px-8 py-3.5 rounded-xl shadow-lg shadow-amber-500/20 hover:brightness-105 active:scale-[0.98] transition-all"
    >
       Run PUT Request (Full Update)
    </button>
  ) : editMethod === "PATCH" ? (
    <button
      onClick={handlePatchUpdate}
      className="w-full md:w-auto bg-gradient-to-r from-indigo-500 to-blue-600 text-white font-semibold px-8 py-3.5 rounded-xl shadow-lg shadow-indigo-500/20 hover:brightness-105 active:scale-[0.98] transition-all"
    >
       Run PATCH Request (Name Only)
    </button>
  ) : (

```



```

        <button
            onClick={handleCreate}
            className="w-full md:w-auto bg-gradient-to-r from-emerald-500 to-teal-600 text-white font-semibold px-8 py-3.5 rounded-xl shadow-lg shadow-emerald-500/20 hover:brightness-105 active:scale-[0.98] transition-all"
        >
            🚀 Run POST Request (Create)
        </button>
    )}
</div>
</div>

{/* --- [UI PART: USER LIST CARDS] --- */}
<div className="max-w-6xl mx-auto">
    {loading ? (
        <div className="text-center py-20">
            <div className="inline-block animate-spin rounded-full h-10 w-10 border-4 border-blue-500 border-t-transparent mb-4"></div>
            <h2 className="text-xl font-semibold text-slate-600">Fetching users from database...</h2>
        </div>
    ) : (
        <div className="grid lg:grid-cols-3 md:grid-cols-2 gap-6">
            {users.map((user) => (
                <div
                    key={user.id}
                    className={`group bg-white rounded-2xl border transition-all duration-300 flex flex-col justify-between overflow-hidden ${
                        editingUserId === user.id
                            ? 'border-blue-500 ring-4 ring-blue-500/10 shadow-xl scale-[1.01]'
                            : 'border-slate-100 shadow-md hover:shado

```

```

w-xl hover:-translate-y-1'
    }` }
  >
    { /* CARD HEADER */ }
    <div className="p-6 pb-4 border-b border-slate-50 bg-gradient-to-b from-slate-50/80 to-transparent">
      <div className="flex items-start justify-between gap-2">
        <div>
          <h2 className="text-xl font-bold text-slate-800 group-hover:text-blue-600 transition-colors line-clamp-1">
            {user.name}
          </h2>
          <p className="text-sm font-medium text-slate-400 mt-0.5">
            @{user.username || "username"}
          </p>
        </div>
        <span className="text-xs bg-slate-100 text-slate-600 px-2 py-1 rounded-md font-bold shrink-0">
          ID: {String(user.id).slice(-4)}
        </span>
      </div>
    </div>

    { /* CARD BODY */ }
    <div className="p-6 pt-4 space-y-3 flex-grow text-sm text-slate-600">
      <div className="flex items-center gap-3">
        <span className="text-base grayscale group-hover:grayscale-0 transition-all">✉</span>
        <span className="truncate" title={user.email}>{user.email || "N/A"}</span>
      </div>
      <div className="flex items-center gap-3">

```

```

        <span className="text-base grayscale group-hover:grayscale-0 transition-all">☎️</span>
        <span className="truncate">{user.phone ||
"N/A"}</span>
    </div>
    <div className="flex items-center gap-3">
        <span className="text-base grayscale group-hover:grayscale-0 transition-all">🌐</span>
        <span className="truncate text-blue-500 hover:underline cursor-pointer">
            {user.website || "N/A"}
        </span>
    </div>
    {user.company && (
        <div className="flex items-center gap-3 p-t-2 border-t border-slate-100/80">
            <span className="text-base grayscale group-hover:grayscale-0 transition-all">🏢</span>
            <span className="truncate font-medium text-slate-500">{user.company.name}</span>
        </div>
    )}
    {user.address && (
        <div className="flex items-center gap-3">
            <span className="text-base grayscale group-hover:grayscale-0 transition-all">📍</span>
            <span className="truncate font-medium text-slate-500">{user.address.city}</span>
        </div>
    )}
</div>

{/* CARD ACTIONS (BUTTONS) */}
<div className="p-4 bg-slate-50 border-t border-slate-100 grid grid-cols-3 gap-2">
    <button

```

```

        onClick={() => startEdit(user, "PUT")}
        className={`border text-xs font-bold py-
2.5 px-2 rounded-xl transition-all active:scale-[0.95] ${
            editingUserId === user.id && editMethod
=== 'PUT'
                ? 'bg-amber-500 text-white border-amb
er-500 shadow-md shadow-amber-500/20'
                : 'bg-white hover:bg-amber-50 border-
slate-200 text-amber-600'
            }`}
    >
        PUT (Edit)
    </button>

    <button
        onClick={() => startEdit(user, "PATCH")}
        className={`border text-xs font-bold py-
2.5 px-2 rounded-xl transition-all active:scale-[0.95] ${
            editingUserId === user.id && editMethod
=== 'PATCH'
                ? 'bg-indigo-500 text-white border-in
digo-500 shadow-md shadow-indigo-500/20'
                : 'bg-white hover:bg-indigo-50 border
-slate-200 text-indigo-600'
            }`}
    >
        PATCH
    </button>

    <button
        onClick={() => handleDelete(user.id)}
        className="bg-red-50 border border-transp
arent hover:bg-red-100 text-red-600 text-xs font-bold py-2.5
px-2 rounded-xl transition-all active:scale-[0.95]"
    >
        DELETE

```

```
        </button>
      </div>

    </div>
  )})
</div>
})
</div>

</div>
);
}
```

6. Project run করো

```
npm run dev
```

Postman Complete Practice Guide (Exam Focused)

তোমার Exam Scope এ Postman-এর জন্য যা যা বলা আছে:

- ☒ Create Collection
- ☒ Send Requests
- ☒ Test Endpoints
- ☒ Add Headers/Body
- ☒ JSON Response Handling
- ☒ Environment Variables Practice
- ☒ GET
- ☒ POST

-  PUT
-  PATCH
-  DELETE

এসব একসাথে Practice করলে Postman Part পুরো Cover হয়ে যাবে।

Step 1: Postman Install

[Postman Official Website](#)

Install করে Login করো।

Step 2: Create Collection

Collection হলো API Request গুলোকে Group করে রাখা।

Create Collection

1. New
2. Collection
3. Name:

API Practice

1. Create

এখন Collection এর ভিতরে সব Request Save করবে।

Step 3: API Practice Website

Practice এর জন্য ব্যবহার করো:

<https://jsonplaceholder.typicode.com>

Step 4: GET Request

GET মানে Data Read করা।

New Request

Method:

GET

URL:

`https://jsonplaceholder.typicode.com/posts`

Click:

Send

Result

Response:

```
[
  {
    "userId":1,
    "id":1,
    "title":"..."
  }
]
```

এটাই JSON Response।

GET By ID

URL:

```
https://jsonplaceholder.typicode.com/posts/1
```

Send

একটা Post Return করবে।

Exam Question হতে পারে

"Fetch post with ID 1"

তুমি GET দিয়ে URL hit করবে।

Step 5: POST Request

POST = New Data Create

Method:

```
POST
```

URL:

```
https://jsonplaceholder.typicode.com/posts
```

Body Tab

Click:

```
Body
```




raw



JSON

Write JSON

```
{  
  "title": "My First Post",  
  "body": "Testing API",  
  "userId": 1  
}
```

Send

Response:

```
{  
  "title": "My First Post",  
  "body": "Testing API",  
  "userId": 1,  
  "id": 101  
}
```

মানে Create হয়েছে।

Step 6: PUT Request

PUT = Entire Data Update

Method:

PUT

URL:

`https://jsonplaceholder.typicode.com/posts/1`

Body:

```
{  
  "id":1,  
  "title":"Updated",  
  "body":"New Body",  
  "userId":1  
}
```

Send

পুরো Object Replace হবে।

Step 7: PATCH Request

PATCH = Partial Update

Method:

PATCH

URL:

`https://jsonplaceholder.typicode.com/posts/1`

Body:

```
{  
  "title": "Only Title Changed"  
}
```

Send

শুধু title update হবে।

PUT vs PATCH

PUT

```
{  
  "id": 1,  
  "title": "ABC",  
  "body": "XYZ"  
}
```

পুরো Resource Update

PATCH

```
{  
  "title": "ABC"  
}
```

শুধু Specific Field Update

Step 8: DELETE Request

Method:

DELETE

URL:

```
https://jsonplaceholder.typicode.com/posts/1
```

Send

Response:

```
{}
```

Status:

200 OK

মানে Delete Request Success।

Step 9: Headers Practice

Headers Tab এ যাও।

Add:

Key	Value
Content-Type	application/json

আরও Example:

Key	Value	
Authorization	Bearer token123	

Exam এ Header Add করতে বলতে পারে।

Example 2: Authorization Header

অনেক API Login ছাড়া Access করা যায় না।

ধরো Facebook বা GitHub API ব্যবহার করবে।

Server বলবে:

| "প্রথমে প্রমাণ করো তুমি কে।"

তখন Token পাঠাতে হয়।

```
Authorization: Bearer abc123xyz
```

এখানে

```
Bearer
```

= Token Type

```
abc123xyz
```

= Access Token

Real Example

```
GET /users
```

```
Authorization: Bearer eyJhbGc...
```

Server Token Verify করে Data দেয়।

Example 3: Accept Header

```
Accept: application/json
```

মানে:

| "Response আমাকে JSON Format-এ দাও।"

Step 10: Query Parameters

URL:

```
https://jsonplaceholder.typicode.com/posts
```

Params Tab:

Key	Value
userId	1

Send

Result:

```
Only user 1 posts
```

Step 11: Status Code Check

Response এর পাশে দেখবে:

```
200 OK
```

Common Status:

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Server Error

Step 12: Response Time Check

Response নিচে দেখাবে:

150 ms

API কত দ্রুত Response দিয়েছে।

Step 13: Save Request

Click:

Save

Collection:

API Practice

সব Request Save করো।

Step 14: Environment Variables

Exam এ খুব Common।

Create Environment

Top Right

Environments



Create New

Name:

Development

Add Variable

Variable	Value
base_url	https://jsonplaceholder.typicode.com

Save

Now URL

Instead of

<https://jsonplaceholder.typicode.com/posts>

Use:

`{{base_url}}/posts`

Send

একই Result।

Step 15: Postman Tests

Tests Tab এ যাও।

Write:

```
pm.test("Status Code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

Send

Result:

PASS

আরও Example

```
pm.test("Response Time", function () {  
    pm.expect(pm.response.responseTime).to.be.below(500);  
});
```

Step 16: Collection Runner

একসাথে সব Request Run করা যায়।

Collection

↓

Run Collection

↓

Run

Exam Practice Task (নিজে করো)

Task 1

GET

```
{{base_url}}/posts
```

Task 2

GET

```
{{base_url}}/posts/1
```

Task 3

POST

```
{
  "title": "Exam",
  "body": "Practice",
  "userId": 1
}
```

Task 4

PUT

```
{
  "id": 1,
  "title": "Updated Exam",
  "body": "Updated",
  "userId": 1
}
```

Task 5

PATCH

```
{
  "title": "Patch Update"
}
```

Task 6

DELETE

```
{{base_url}}/posts/1
```

তুমি যে ২টা Topic বলছো:

1. **Selenium IDE**
2. **Selenium WebDriver**

এদের কাজ একই Goal-এর জন্য: **Software Automation Testing**, কিন্তু কাজ করার পদ্ধতি আলাদা।

1 Selenium IDE

কী কাজে ব্যবহার হয়?

Manual Testing-কে Automatic করার জন্য।

ধরো Website-এ প্রতিদিন একই কাজ করতে হয়:

- Login
- Search
- Click
- Logout

প্রতিদিন হাতে করার বদলে Selenium IDE Record করে রাখে।

Real Example

তুমি Browser-এ:

1. Google Open করলে
2. "Python" লিখলে
3. Search করলে

Selenium IDE এগুলো Record করবে।

পরে Run দিলে নিজে নিজে করবে।

IDE-এর সুবিধা

- ✓ Coding লাগে না
 - ✓ Beginners Friendly
 - ✓ Quick Testing
 - ✓ Record & Playback
-

Exam-এ কী করতে বলতে পারে?

| Record a test case that searches "Python" in Google.

তুমি Record করে Run দিবে।

2 Selenium WebDriver

কী কাজে ব্যবহার হয়?

Code লিখে Browser Control করা।

এটা Professional Automation Testing-এর জন্য ব্যবহার হয়।

Real Example

Python Code:

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://google.com")
```

Code Run করলে Browser Open হবে।

আরও কাজ

```
search.send_keys("Python")
```

নিজে নিজে টাইপ করবে।

```
button.click()
```

নিজে নিজে Click করবে।

```
assert "Python" in driver.title
```

Check করবে Result ঠিক আছে কিনা।

Selenium IDE vs WebDriver

Selenium IDE	Selenium WebDriver
No Coding	Coding লাগে
Record & Play	Full Automation
Easy	Powerful
Small Testing	Large Project
Beginner	Professional

Real Industry-তে কোনটা বেশি ব্যবহার হয়?

প্রায় সব Software Company:

✓ Selenium WebDriver

✓ Pytest

ব্যবহার করে।

Selenium IDE সাধারণত Learning বা Quick Demo-এর জন্য।

Exam Perspective

যদি Sir বলে:

| "Record and Run a Test"

➡ Selenium IDE

যদি Sir বলে:

| "Write automation script"

➡ Selenium WebDriver

Example:

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

driver.get("https://google.com")

driver.find_element(
    By.NAME,
    "q"
).send_keys("Python")

driver.quit()
```

সহজে মনে রাখো

Selenium IDE

👤 Screen Recorder-এর মতো

তুমি যা করো সেটা Record করে।

Selenium WebDriver

Robot-এর মতো

তুমি Code লিখে Robot-কে বলো কী করতে হবে, Robot Browser-এ সেটা করে।

তোমার Exam Scope দেখে আমি বলব **WebDriver + Pytest** অংশে বেশি Focus দাও, কারণ Live Coding Exam-এ Script লিখতে দেওয়ার সম্ভাবনা Selenium IDE-এর চেয়ে বেশি।

Selenium WebDriver Full Process (Exam-Oriented)

Selenium WebDriver হলো এমন একটি Tool যা **Python Code** দিয়ে **Browser Control** করে।

তুমি Code লিখবে, Selenium Browser-এ গিয়ে সেই কাজ করবে।

Step 1: Python Install Check

Terminal/CMD:

```
python --version
```

Output:

```
Python 3.x.x
```

Step 2: Selenium Install

CMD:

```
pip install selenium
```

Check:

```
pip show selenium
```

Step 3: VS Code/Open IDE

নতুন File:

```
test.py
```

Step 4: First Selenium Program

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://www.google.com")

print(driver.title)

driver.quit()
```

Run:

```
python test.py
```

কী হবে?

1. Chrome Open হবে
2. Google Open হবে
3. Google Title Print হবে
4. Browser Close হবে

Step 5: Find Element

Website-এর Element খরতে হয়।

Example:

Google Search Box

```
from selenium.webdriver.common.by import By

search = driver.find_element(
    By.NAME,
    "q"
)
```

Common Locators

Locator	Example
ID	By.ID
NAME	By.NAME
XPATh	By.XPATh
CLASS_NAME	By.CLASS_NAME
TAG_NAME	By.TAG_NAME
LINK_TEXT	By.LINK_TEXT

Step 6: Type Text

```
search.send_keys("Python Tutorial")
```

Browser নিজে টাইপ করবে।

Step 7: Submit Search

```
search.submit()
```

অথবা

```
from selenium.webdriver.common.keys import Keys

search.send_keys(Keys.ENTER)
```

Full Google Search Example

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

driver.get("https://www.google.com")
time.sleep(3)
search = driver.find_element(
    By.NAME,
    "q"
)

search.send_keys("Python Tutorial")

search.submit()

driver.quit()
```

Step 8: Click Button

```
button = driver.find_element(  
    By.ID,  
    "submit"  
)  
  
button.click()
```

Step 9: Wait

Website Load হতে Time লাগে।

```
import time  
  
time.sleep(3)
```

Better:

```
driver.implicitly_wait(10)
```

মানে 10 Second পর্যন্ত অপেক্ষা করবে।

Step 10: Assertion

Result Verify করা।

```
assert "Python" in driver.title
```

Example

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

driver.get("https://www.google.com")

search = driver.find_element(
    By.NAME,
    "q"
)

search.send_keys("Python")

search.submit()

assert "Python" in driver.title

driver.quit()
```

Login Automation Example

ধরো Login Form আছে।

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://opensource-demo.orangehrmlive.com/web/index.php/auth/login")

time.sleep(5)
```

```
driver.find_element(  
    By.NAME,  
    "username"  
) .send_keys("Admin")  
  
driver.find_element(  
    By.NAME,  
    "password"  
) .send_keys("admin123")  
  
driver.find_element(  
    By.TAG_NAME,  
    "button"  
) .click()  
  
time.sleep(5)  
  
driver.quit()
```

Exam Practice 1

Wikipedia Search

```
from selenium import webdriver  
from selenium.webdriver.common.by import By  
import time  
  
driver = webdriver.Chrome()  
  
driver.get("https://www.wikipedia.org")  
  
search = driver.find_element(  
    By.ID,
```

```
        "searchInput"
    )

    search.send_keys("Bangladesh")

    search.submit()

    time.sleep(3)

    assert "Bangladesh" in driver.title

    print("Test Passed")

    driver.quit()
```

Exam Practice 2

Verify Title

```
assert "Bangladesh" in driver.title
```

Exam-এর জন্য **Open Website + Search + Click + Wait + Assertion + Close Browser** সব একসাথে আছে এমন Complete Selenium WebDriver Code নিচে দিলাম।

Wikipedia Search Automation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# Open Chrome Browser
driver = webdriver.Chrome()

# Wait up to 10 seconds for elements
```

```
driver.implicitly_wait(10)

# Open Website
driver.get("https://www.wikipedia.org")

# Verify Website Opened
assert "Wikipedia" in driver.title

# Find Search Box
search = driver.find_element(By.ID, "searchInput")

# Type Text
search.send_keys("Bangladesh")

# Submit Search
search.submit()

# Wait to see result
time.sleep(3)

# Verify Result Page
assert "Bangladesh" in driver.title

print("Test Passed Successfully!")

# Close Browser
driver.quit()
```

Login Automation Example (OrangeHRM)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
```

```

driver = webdriver.Chrome()

driver.implicitly_wait(10)

driver.get(
    "https://opensource-demo.orangehrmlive.com/web/index.php/auth/login"
)

# Wait for page load
time.sleep(5)

# Enter Username
driver.find_element(
    By.NAME,
    "username"
).send_keys("Admin")

# Enter Password
driver.find_element(
    By.NAME,
    "password"
).send_keys("admin123")

# Click Login
driver.find_element(
    By.TAG_NAME,
    "button"
).click()

time.sleep(5)

# Verify Login Success
assert "dashboard" in driver.current_url.lower()

print("Login Successful!")

```



```
driver.quit()
```

Pytest Version (Exam Favourite)

Install করো:

```
pipinstallpytest
```

তারপর:

```
python-mpytest
```

File Name:

```
test_wikipedia.py
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

def test_wikipedia():

    driver = webdriver.Chrome()

    driver.get("https://www.wikipedia.org")

    search = driver.find_element(
        By.ID,
        "searchInput"
    )
```

```
search.send_keys("Bangladesh")

driver.find_element(
    By.CSS_SELECTOR,
    "button[type='submit']"
).click()

time.sleep(3)

print(driver.title)

assert "Bangladesh" in driver.title

driver.quit()
```

Run:

```
python -m pytest
```

Pytest + HTML Report

Step 1: Project Folder Create

Folder:

```
C:\Users\PC\Documents\selenium_practice
```

VS Code দিয়ে Folder Open করো।

Step 2: Install Packages

Terminal Open:

```
pip install selenium
```

```
pip install pytest
```

```
pip install pytest-html
```

Check:

```
python-m pytest--version
```

Step 1: Project Folder

```
Palindrome_Project
```

Step 2: Create File

File Name:

```
palindrome.py
```

Code:

```
def is_palindrome(text):  
  
    text = text.lower()  
  
    return text == text[::-1]
```

Explanation

Input:

```
Racecar
```

Lowercase:

```
racecar
```

Reverse:

```
racecar
```

Compare:

```
racecar == racecar
```

Output:

```
True
```

Step 3: Create Test File

File Name:

```
test_palindrome.py
```

Code:

```
from palindrome import is_palindrome

def test_valid_palindrome():
```

```
assert is_palindrome("madam") == True

def test_non_palindrome():

    assert is_palindrome("python") == False

def test_mixed_case_palindrome():

    assert is_palindrome("Racecar") == True
```

Step 4: Folder Structure

```
Palindrome_Project
├── palindrome.py
└── test_palindrome.py
```

Step 5: Install pytest

Terminal:

```
python -m pip install pytest
```

Check:

```
python -m pytest --version
```

Step 6: Run pytest

Project Folder-এর Terminal থেকে:

```
python -m pytest
```

Expected Output:

```
===== test session starts =====  
  
collected 3 items  
  
test_palindrome.py ... [100%]  
  
===== 3 passed =====
```

Step 7: Install pytest-html

```
python -m pip install pytest-html
```

Step 8: Generate HTML Report

```
python -m pytest --html=report.html
```

Output:

```
generated html report: file:///.../report.html
```

Step 9: Open Report

Project Folder-এ:

```
report.html
```

File তৈরি হবে।

Double Click করো।

Browser-এ Report দেখাবে:

PASSED
PASSED
PASSED

Bonus Edge Case Version (Teacher Impress 😊)

```
from palindrome import is_palindrome

def test_valid_palindrome():
    assert is_palindrome("madam") == True

def test_non_palindrome():
    assert is_palindrome("python") == False

def test_mixed_case_palindrome():
    assert is_palindrome("Racecar") == True

def test_empty_string():
    assert is_palindrome("") == True

def test_single_character():
    assert is_palindrome("a") == True
```

Run:

```
python -m pytest
```

Output:

```
5 passed
```

Exam Submission Ready Files

palindrome.py

```
def is_palindrome(text):  
  
    text = text.lower()  
  
    return text == text[::-1]
```

test_palindrome.py

```
from palindrome import is_palindrome  
  
def test_valid_palindrome():  
  
    assert is_palindrome("madam") == True  
  
def test_non_palindrome():  
  
    assert is_palindrome("python") == False
```



```
def test_mixed_case_palindrome():  
    assert is_palindrome("Racecar") == True
```

Commands

```
python -m pytest
```

```
python -m pytest --html=report.html
```